1. **Aim/Overview of the practical:** Create a Signup page and store the inputted data by user in file.
2. **Objective of the practical:** To create a Signup page and store user data securely in a JSON file using Node.js and the File System (fs) module.

3. **Code for experiment/practical:**

```
import React, { useState } from "react";
import "./styles/form.css";

const Login = ({ onSwitch }) => {
  const [formData, setFormData] = useState({ email: "", password: "" });

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await fetch("http://localhost:5000/api/auth/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formData),
    });
    const data = await res.json();
    alert(data.message);
  };

  return (
```

```
    <div className="signup-container">
      <h2>Login</h2>
      <form onSubmit={handleSubmit}>
        <input type="email" name="email" placeholder="Email" onChange={handleChange}
required />
        <input type="password" name="password" placeholder="Password"
onChange={handleChange} required />

        <button type="submit">Login</button>
      </form>
      <p>Don’t have an account? <span className="link"
onClick={onSwitch}>Signup</span></p>
    </div>
  );
};

export default Login;
```

Server.js

```
import express from "express";
import cors from "cors";
import bodyParser from "body-parser";
import authRoutes from "./routes/auth.js";

const app = express();
const PORT = 5000;

app.use(cors());
app.use(bodyParser.json());

// Routes
app.use("/api/auth", authRoutes);

app.listen(PORT, () => {
  console.log(` Server running on http://localhost:${PORT}`);
});
```

Auth.js

```
import express from "express";
import fs from "fs";
import path from "path";
import bcrypt from "bcrypt";

const router = express.Router();
const usersFilePath = path.join(process.cwd(), "users.json");

//  Signup
router.post("/signup", async (req, res) => {
  const { firstName, lastName, email, password, gender, course, address } = req.body;

  // check if email already exists
  let users = [];
  if (fs.existsSync(usersFilePath)) {
    users = JSON.parse(fs.readFileSync(usersFilePath, "utf-8"));
  }
  const userExists = users.find((u) => u.email === email);
  if (userExists) {
    return res.status(400).json({ message: "User already exists!" });
  }

  //  Hash the password before saving
  const hashedPassword = await bcrypt.hash(password, 10); // 10 = salt rounds

  const newUser = {
    firstName,
    lastName,
    email,
    password: hashedPassword,
    gender,
    course,
    address,
  };

  users.push(newUser);
  fs.writeFileSync(usersFilePath, JSON.stringify(users, null, 2));

  res.json({ message: "Signup successful!" });
});
```

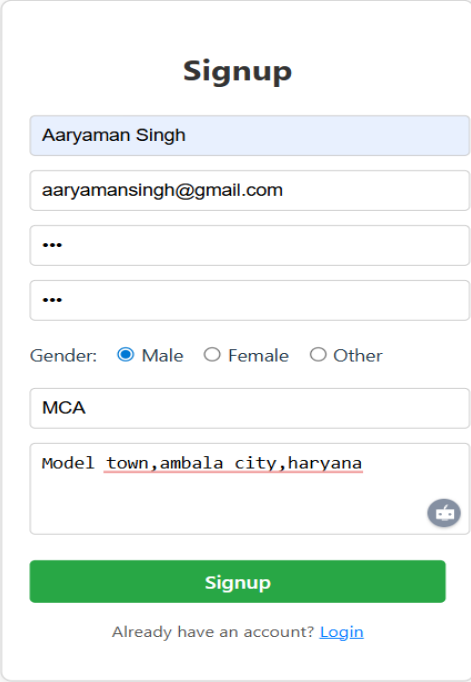
```
// Login
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  if (!fs.existsSync(usersFilePath)) {
    return res.status(400).json({ message: "No users found!" });
  }

  const users = JSON.parse(fs.readFileSync(usersFilePath, "utf-8"));
  const user = users.find((u) => u.email === email);
  if (!user) {
    return res.status(400).json({ message: "Invalid email or password!" });
  }

  // Compare password with hash
  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) {
    return res.status(400).json({ message: "Invalid email or password!" });
  }
  res.json({ message: "Login successful!" });
});
export default router;
```

4. Output:




```
[
  {
    "email": "dimpal@gmail.com",
    "password": "$2b$10$rfNCy.j7tzXiDTe0goANWu8GJYT0w8VSLvu1o64MCl74c9CsRvSLi",
    "gender": "Male",
    "course": "MCA",
    "address": "#159,Kharar,Mohali"
  },
  {
    "email": "aaryamansingh49@gmail.com",
    "password": "$2b$10$STTD8IOkHAMY73zDsXAAMu3AWr9lATMr/0M2Lgte2djoQfF9NSM.i",
    "gender": "Male",
    "course": "MCA",
    "address": "#206,Model Town, Ambala city, Haryana."
  },
  {
    "email": "ankush@gmail.com",
    "password": "$2b$10$cXIW2yHn80.QnfTaByghJubwi8KUp.OuNfzqibAEQtM8CAob4UH.e",
    "gender": "Male",
    "course": "MCA",
    "address": "500,kharar,mohali"
  },
  {
    "email": "aaryamansingh@gmail.com",
    "password": "$2b$10$x.nK7O/alISeSWPBgZ0AVumHWKzg36FCQ25voa6ZOQl3fIOwh3xki",
    "gender": "Male",
    "course": "MCA",
    "address": "206.model town,ambala city,haryana"
  }
```


5. Learning Outcomes (What I have learnt):

- Students will learn how to design a signup form and capture user input effectively.
- They will understand how to use Node.js's File System (fs) module to read, write, and update JSON files.
- They will gain practical knowledge of handling user data storage without databases for small-scale applications.